

Deploying the frontend to Vercel

The React frontend deploys to Vercel on the free tier. **The backend cannot go there** — Ollama needs a 5.2 GB model resident in RAM and 3–10s of inference per turn, against a serverless limit of ~250 MB bundle and a 10–60s timeout. That is a platform mismatch, not a configuration problem.

So the working arrangement is:

```
Browser → Vercel (static React) → public tunnel URL
```

Vercel project settings

Setting	Value
Framework Preset	Vite
Root Directory	frontend ← the build fails without this
Build Command	npm run build
Output Directory	dist
Install Command	npm install

Environment variable

Only one, under **Settings** → **Environment Variables**:

Name	Value	Environments
------	-------	--------------

VITE_API_BASE_URL	your public backend URL	Production, Preview, Development
-------------------	----------------------------	-------------------------------------

Rules that matter:

- **No trailing slash.** `https://example.com`, not `https://example.com/`
- **Must be `https://`.** A browser on an HTTPS page blocks requests to `http://`
- **Redeploy after changing it.** Vite inlines `VITE_*` variables at build time, so an env change does nothing until you rebuild:
Deployments → ... → **Redeploy**

⚠ Never put a secret in a `VITE_*` variable. They are compiled into the public JavaScript bundle and readable by anyone. This one is safe because it is only a URL. `GEMINI_API_KEY` and every other credential stay on the backend.

Exposing the local backend

Free: quick tunnel

```
cloudflared tunnel --url http://localhost:8000
```

Prints a `https://<random-words>.trycloudflare.com` URL.
Use that as `VITE_API_BASE_URL`.

Limitations, stated plainly:

- The URL **changes every time you restart the tunnel**. Each restart means updating the Vercel variable and redeploying.
- It dies when the process stops or the Mac sleeps.
- No authentication — anyone with the URL can reach your API.

Fine for a scheduled demo. Not fine for leaving up.

Stable: named tunnel (requires a Cloudflare account and a domain)

```
cloudflared tunnel login
cloudflared tunnel create garden-city
cloudflared tunnel route dns garden-city api.yourdomain.com
cloudflared tunnel run --url http://localhost:8000 garden-city
```

Gives a permanent `https://api.yourdomain.com` — set it once in Vercel and never touch it again.

Backend CORS

The backend rejects browser requests from origins it does not know. Add your Vercel domain to `.env` and restart:

```
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173,
```

Vercel preview deployments get their own generated subdomains. If you need those too, add each one, or run previews against a local backend instead.

Verifying a deployment

Run these against your tunnel URL before blaming the frontend:

```
URL=https://your-tunnel-url

# 1. Is the backend reachable at all?
curl -s "$URL/api/health/live"
# expected: {"status":"alive"}
```

```
# 2. Does CORS allow your Vercel origin? (the usual failure)
curl -s -i -X OPTIONS "$URL/api/chat" \
  -H "Origin: https://your-project.vercel.app" \
  -H "Access-Control-Request-Method: POST" \
  | grep -i access-control-allow-origin
# expected: your Vercel domain echoed back

# 3. Does a real question work?
curl -s -X POST "$URL/api/chat" -H 'Content-Type: application/json' \
  -d '{"message": "When is garbage collection?", "channel": "general"}'
```

In the browser: open the Vercel URL, open DevTools → Network, ask a question. A 200 on `/api/chat` means it is wired correctly.

Troubleshooting

Symptom	Cause	Fix
Blank page, 404 on assets	Root Directory not set to <code>frontend</code>	Fix in project settings and redeploy
"Could not reach the Garden City service"	<code>VITE_API_BASE_URL</code> unset, or tunnel down	Check the variable; confirm the tunnel is running
CORS error in console	Vercel domain not in <code>CORS_ORIGINS</code>	Add it to <code>.env</code> , restart the backend
Mixed-content blocked	URL is <code>http://</code>	Tunnels give HTTPS; use it
Worked yesterday, broken today	Quick-tunnel URL changed on restart	Update the variable, redeploy — or use a named tunnel

Mic button does nothing	Browser blocked it	Allow the microphone; typing always works
Answers arrive but no voice	TTS request failing over the tunnel	Check <code>/api/voice/synthesize</code> directly

Security note

This arrangement has **no authentication on any endpoint**, including the admin routes that delete conversations and approve knowledge entries. A public tunnel URL exposes all of it to anyone who has the link.

Acceptable for a short demo with no real resident data. Before anything longer, see [SECURITY ROADMAP.md](#) — authentication is the first item on that list for exactly this reason.